Disclaimer: These slides can include material from different sources. I'll be happy to explicitly acknowledge a source if required. Contact me for requests.

Machine Learning in a Nutshell

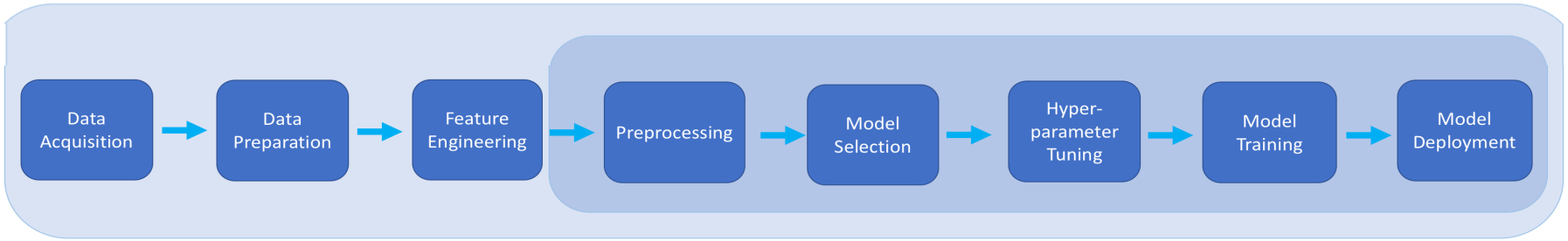
15-288 Spring '26

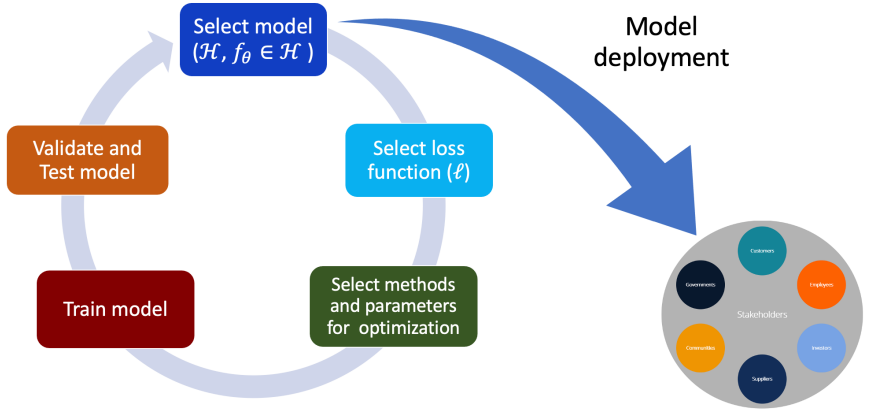
Lecture 5:
Workflow for Regression, Overfitting, Cross-Validation

Teacher:
Gianni A. Di Caro

جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

ML Design for the Canonical problem





- We need to specify:
  1. **The hypothesis class** $\mathcal{H}, f_\theta \in \mathcal{H}$ [Inductive bias, general hypothesis about the mapping]
  2. **The loss function** ℓ [Performance assessment]
  3. **The algorithm for solving the optimization problem** (often approximately) [Compute $\boldsymbol{\theta}$]
  4. **Procedure to create validation sets and to estimate generalization error** [Model selection]

Quiz Time

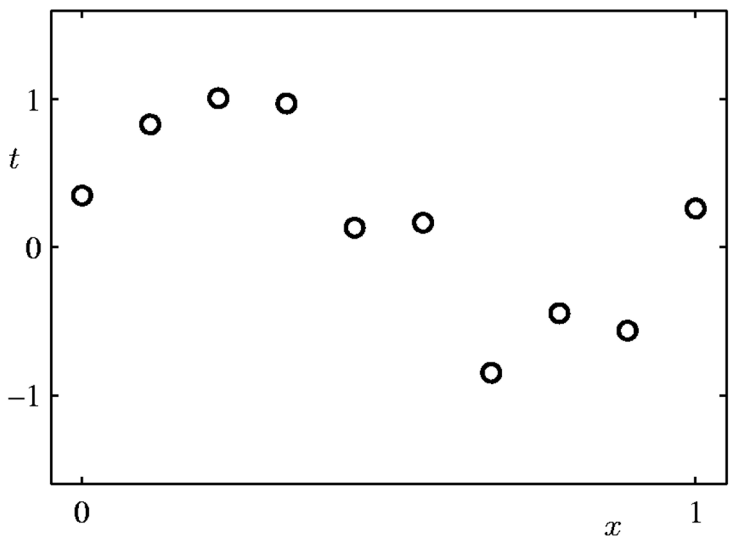
1. I'm given a dataset for an SL task. Assuming the dataset is 'clean', i.e., ready-to-use, what should I do as a very first step toward the development of an effective model for the task?
2. We need to make a hypothesis about the class of functions/hypotheses $\mathcal{H}$ that are appropriate for tackling the task. Is this part of what we have indicated as *inductive bias*?
3. My learned model f_θ can classify correctly all but one example from the training dataset.
   Is the model consistent or not?
4. In general, only one consistent hypothesis is possible, or there could be infinitely many?
5. We're given a training dataset for building a model for predicting the development of diabetes based on a set of 50 biochemical and biometric predictors (i.e., features). The dataset includes 1500 labeled examples. The *empirical error* (i.e., the average error on the training dataset) is 0.05.
   [This error is either the same or a very good approximation of the *generalization error.* We can then safely deploy the model.] Is the statement between [], True or False?
6. Can we precisely compute the generalization error of a learned model? What is the generalization error?
7. All SL problems boil down to solve an optimization problem. Can you write it down / explain it?
8. How can we promote generalization when building an ML model?

Regression example for the workflow

Regression Task:

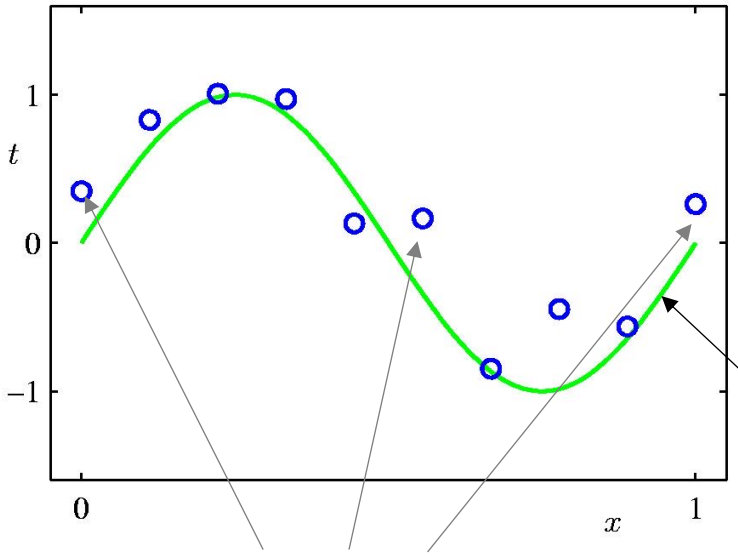
- Learn the curve $f: X \rightarrow T$ that relates the given (x, t) pairs

- $x \in [0,1]$ is a spatial coordinate that indicates the position of a rotating device (e.g., a robotic arm, a motor wheel)
- $t \in [-1,1]$ is value of a measured property of the device at that position



The **training data** in the (x, t) space

Regression example for the workflow: The dataset



$D = \{(x^{(i)}, t^{(i)})\}_{i=1}^{m}, \; m = 10$

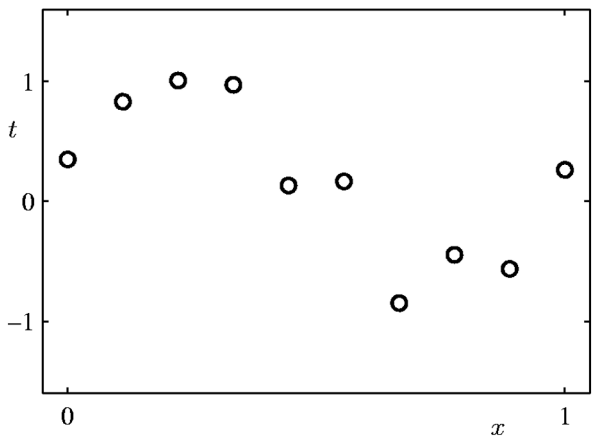
The *true*, unknown, function f is $f(x) = \sin(2\pi x)$

- In this specific case, data points in D have been generated from f by adding a **random noise**

$$D = \{(0.00, 0.47),\ (0.12, 0.89),\ (0.23, 0.99),\ (0.35, 0.97),\ (0.46, 0.12),\ (0.58, 0.13),\ (0.66, -0.92),\ (0.78, -0.62),\ (0.87, -0.48),\ (0.99, 0.23)\}$$

Hypothesis class

This is what we *see by EDA*



Hypothesis: $\mathcal{H}$ *is the set of polynomial* functions of degree M (additive model, linear in the parameters θ)

$$f_\theta(x) = f(x;\boldsymbol{\theta}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots + \theta_M x^M = \sum_{i=0}^{M} \theta_i x^i$$

$\boldsymbol{\theta} \in \mathbb{R}^{M+1}$, $x \in [0,1]$

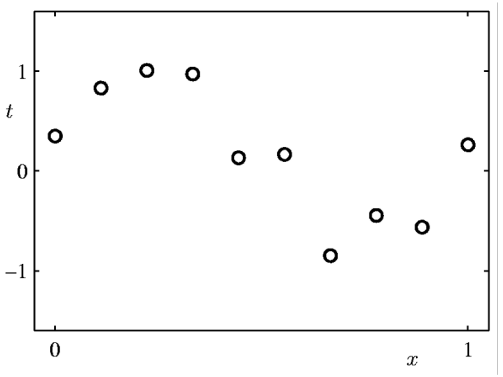
- ✓ M sets the *complexity / capacity of the hypothesized model*
- ✓ M sets the number of parameters $\boldsymbol{\theta}$ to learn → **Difficulty of *task for learning machine***

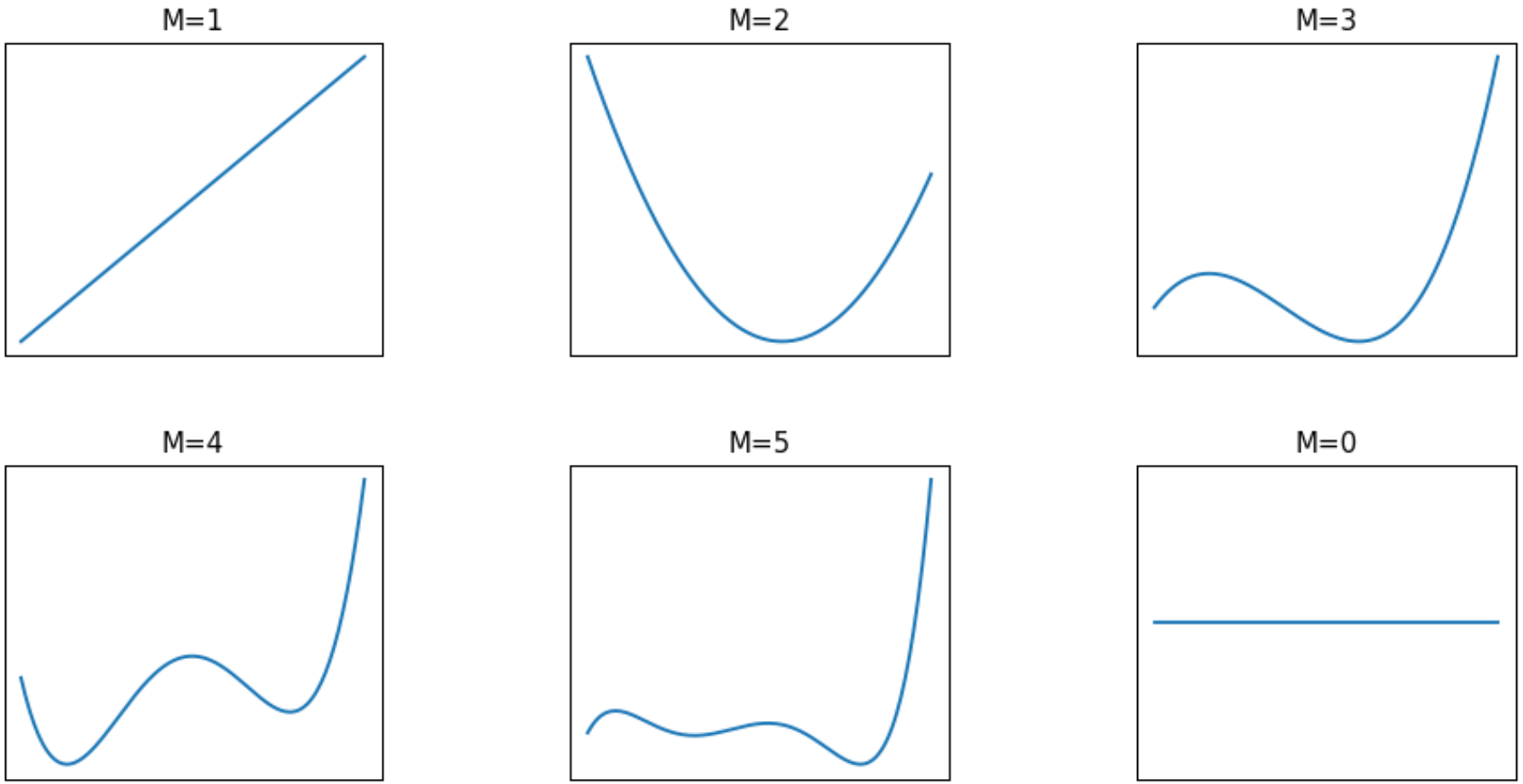
$f_\theta(x) \rightarrow$ For the point of view of ML: $f(\boldsymbol{\theta};\boldsymbol{x}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots + \theta_M x^M$

$\theta_1, \theta_1, \theta_2, \ldots, \theta_{\mathrm{M}}$ → Variables/Parameters of the ML regression task → **Linear** model

Hypothesis class: polynomials

$$f_\theta(x) = f(x;\boldsymbol{\theta}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots + \theta_M x^M = \sum_{i=0}^{M} \theta_i x^i$$





Which model *complexity*?

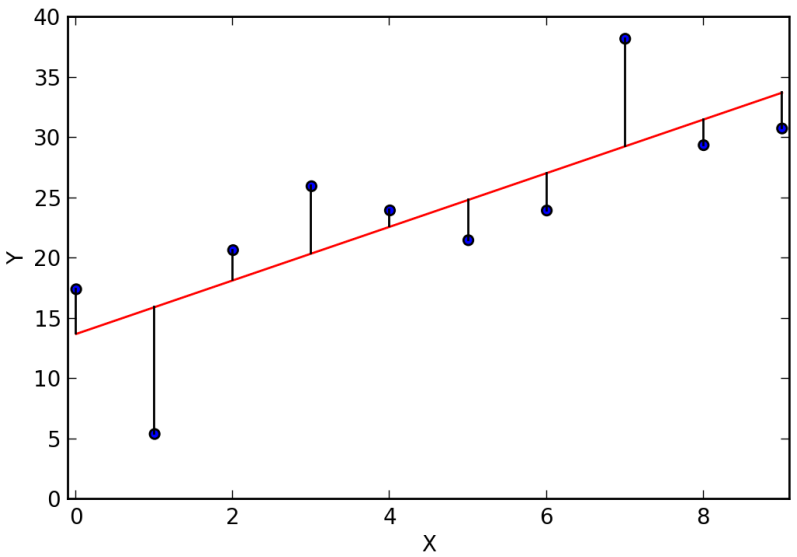
Loss function and Empirical error

Performance / Loss functions:

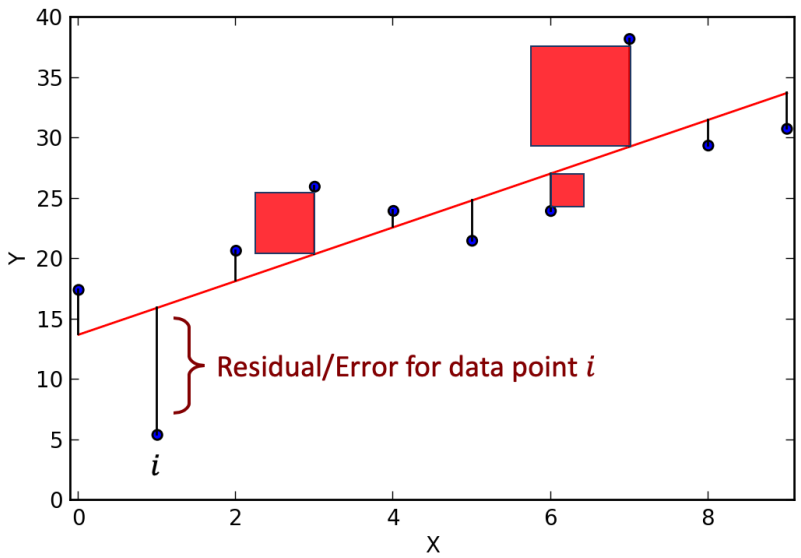
❖ **Sum of Squared Errors (SSE):** $$SSE(\theta) = \frac{1}{2}\sum_{k=1}^{m} \overbrace{\left(f_\theta\left(x^{(k)}\right) - t^{(k)}\right)^2}^{\ell}$$

Empirical error L_{emp}

❖ **Root-Mean-Square Error (RMSE):** $$\boxed{RMSE(\theta) = \sqrt{\frac{SSE(\theta)}{m}}}$$



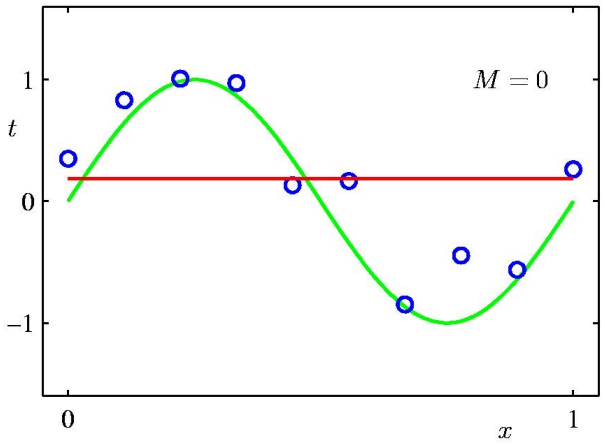
Example data

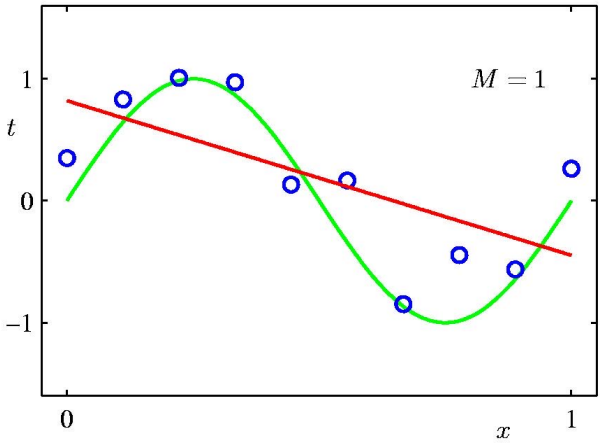


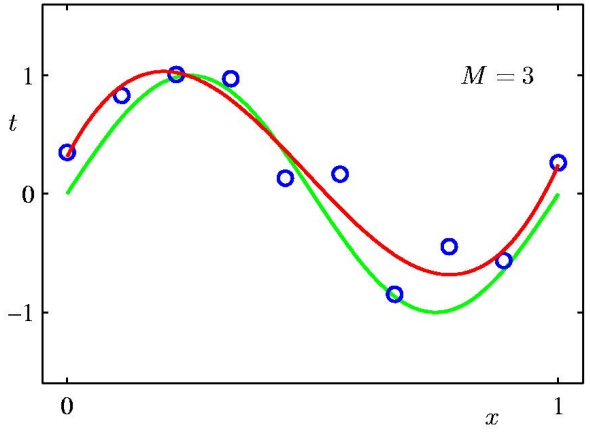
Training (by minimizing the empirical loss)

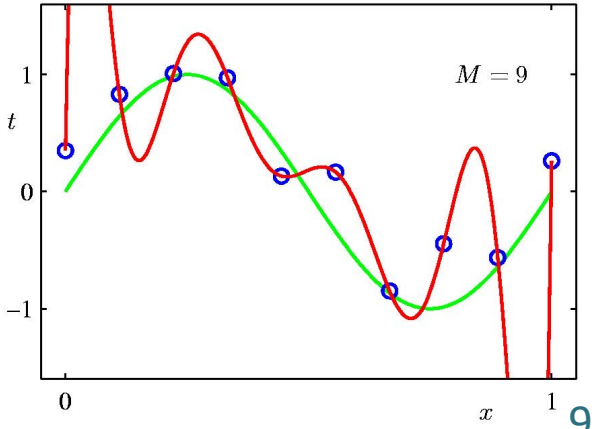
$L_{emp} \equiv$ **Root-Mean-Square Error (RMSE)**

Resulting polynomial models learned by using the training data and RMSE for different model complexity: $M = 0, 1, 3, 9$









RMSE calculation

$L_{emp} \equiv$ **Root-Mean-Square Error (RMSE)**

$$RMSE(\theta) = \sqrt{\frac{SSE(\theta)}{m}} \qquad SSE(\theta) = \frac{1}{2}\sum_{k=1}^{m}\left(f_\theta\left(x^{(k)}\right) - t^{(k)}\right)^2$$

$$f_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots + \theta_M x^M$$

$$D = \{(0.00, 0.47),\ (0.12, 0.89\ ),\ (0.23, 0.99),\ (0.35, 0.97),\ (0.46, 0.12),\ (0.58, 0.13),\ (0.66, -0.92),\ (0.78, -0.62\ ),\ (0.87, -0.48),\ (0.99, 0.23)\}$$

E.g., for $M = 0$

Result of training: $f_{M=0}(x) = 0.22$ (for all x points)

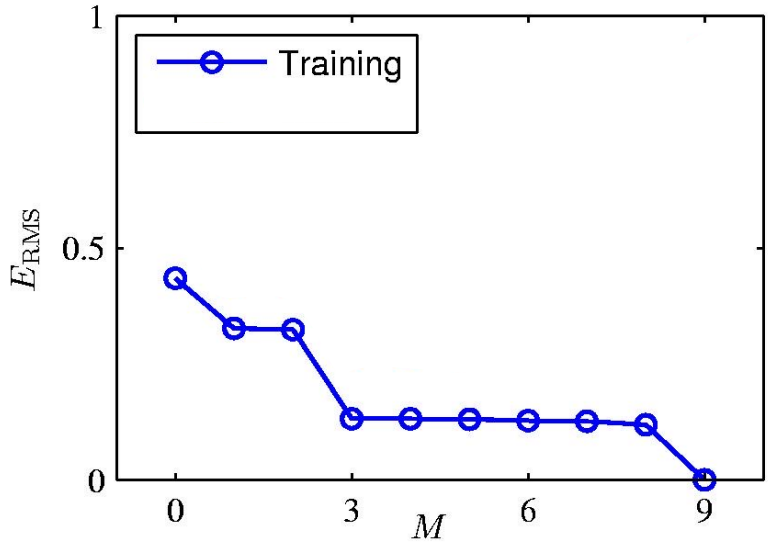
$$RMSE(M=0) = \sqrt{\frac{1}{2\cdot 10}\left(\begin{array}{c}(0.47-0.22)^2 + (0.89-0.22)^2 + (0.99-0.22)^2 + (0.97-0.22)^2 + (0.12-0.22)^2 + \\ (0.13-0.22)^2 + (-0.92-0.22)^2 + (-0.62-0.22)^2 + (-0.48-0.22)^2 + (0.23-0.22)^2\end{array}\right)} \approx 0.46$$

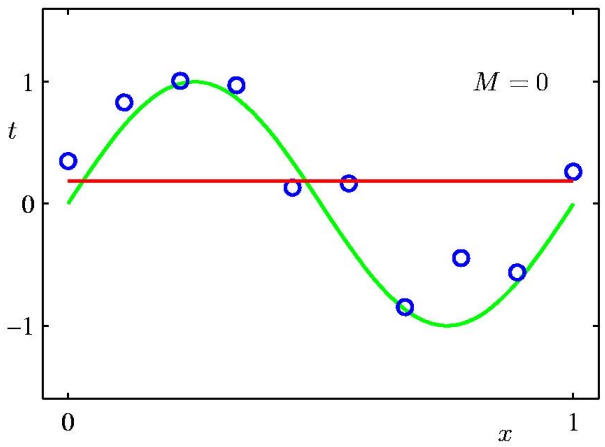
Quiz time

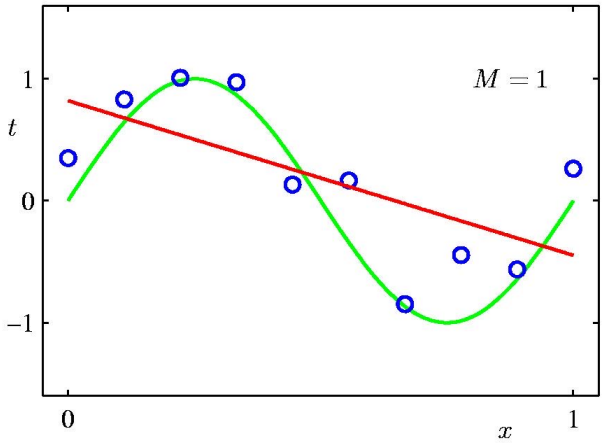
- How many parameters do we have to learn for each one the considered cases?
  - $M = 0$
  - $M = 1$
  - $M = 3$
  - $M = 9$
- Why we haven't considered the cases for $M = 2$, or $M = 4$?
- Why did we stop to $M = 9$? Does it make sense to consider, e.g., $M = 12$?
- Why did we choose RMSE for the empirical error instead of SSE?
- Which model do you think has the minimal value of L_{emp}?

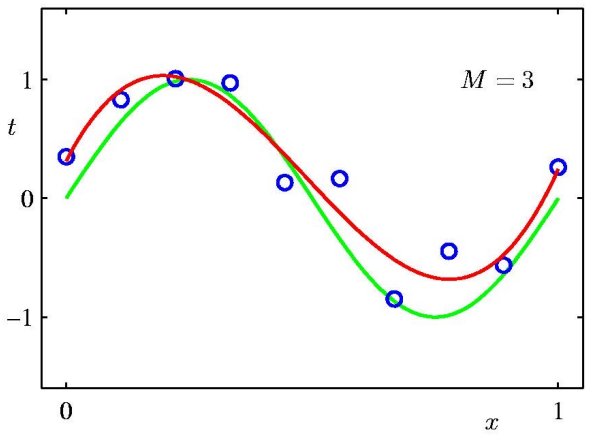
Empirical loss for the different models

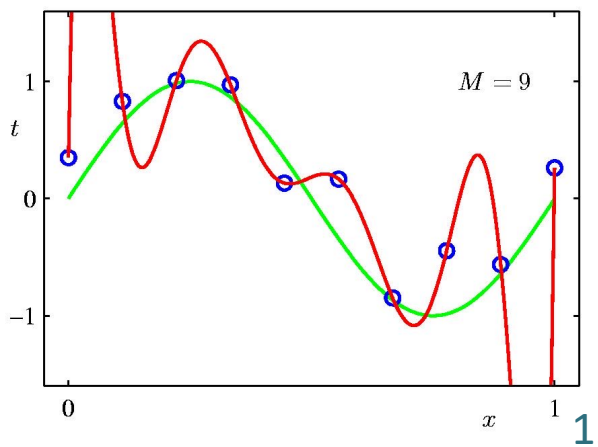
$L_{emp} \equiv$ **Root-Mean-Square Error (RMSE)**



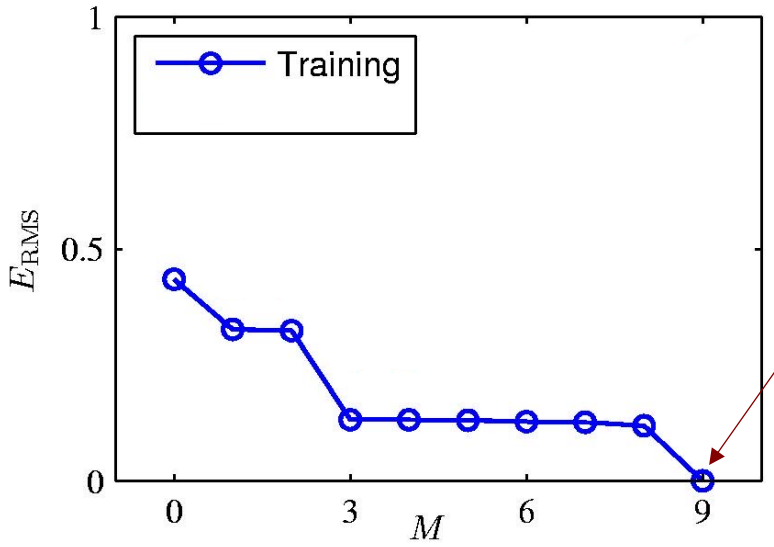






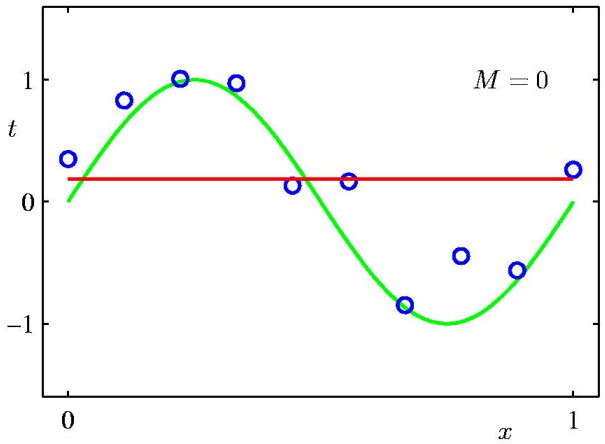


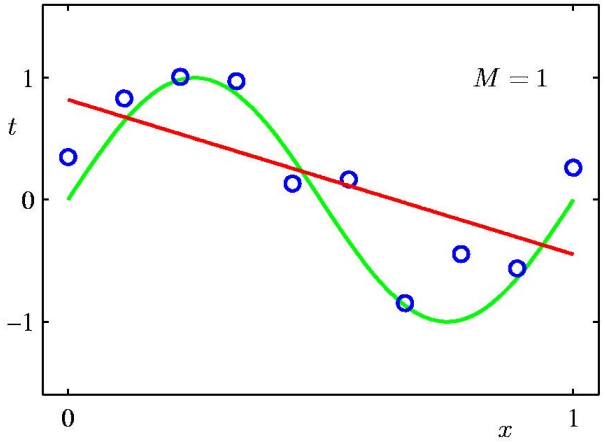
Model choice

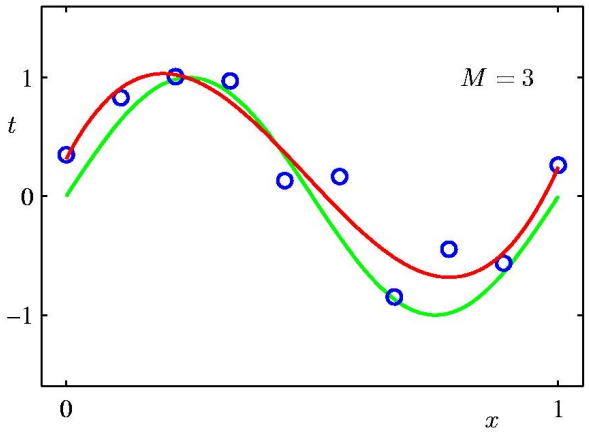


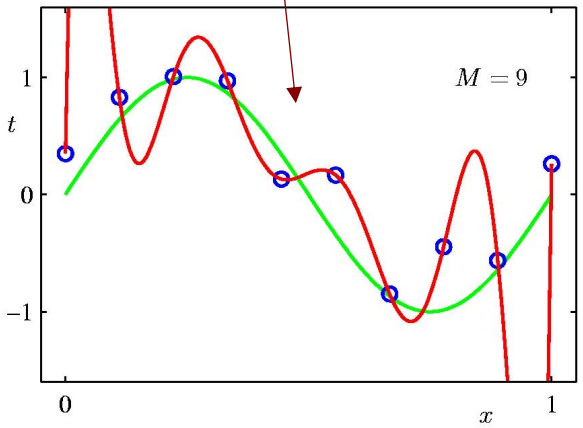
L_{emp} is minimized by the model with $M = 9$

... should we then select the polynomial of degree 9 as model? 🤔

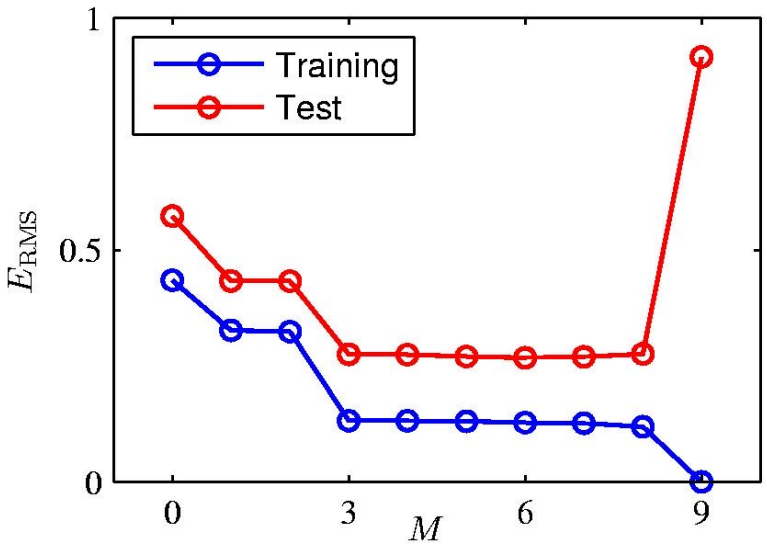


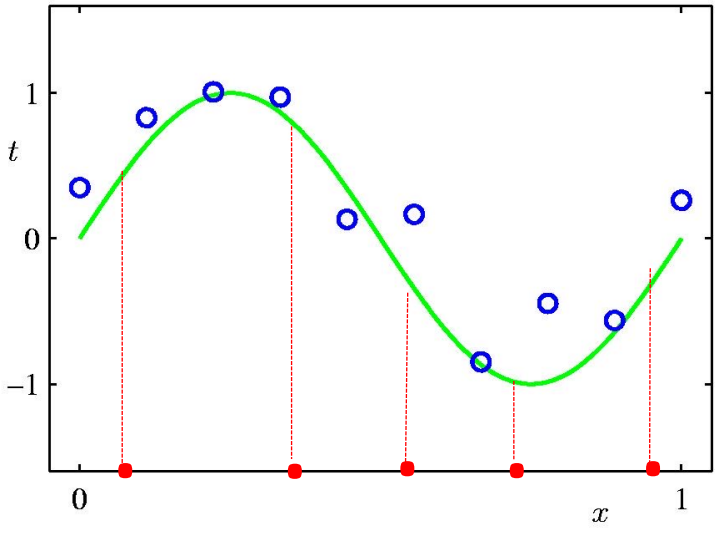




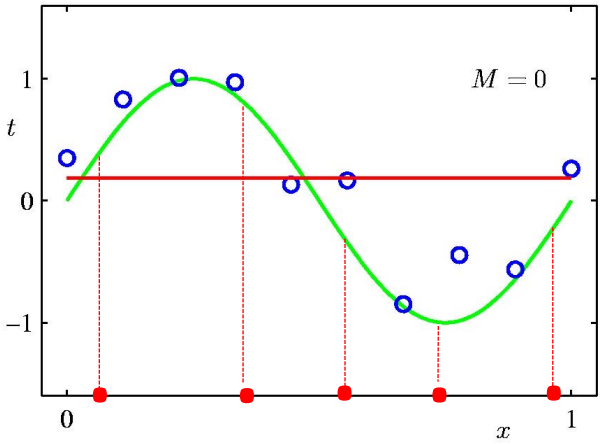


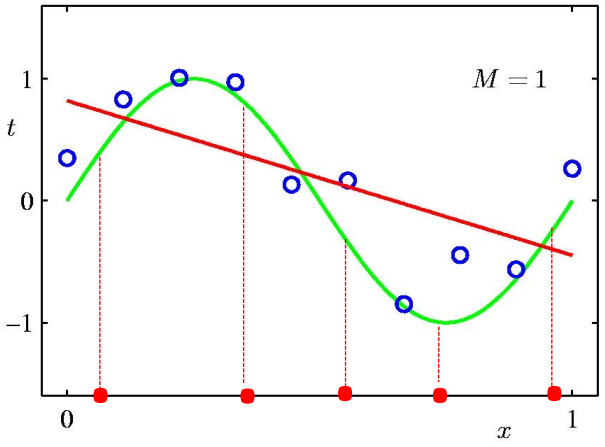
Training vs. Testing

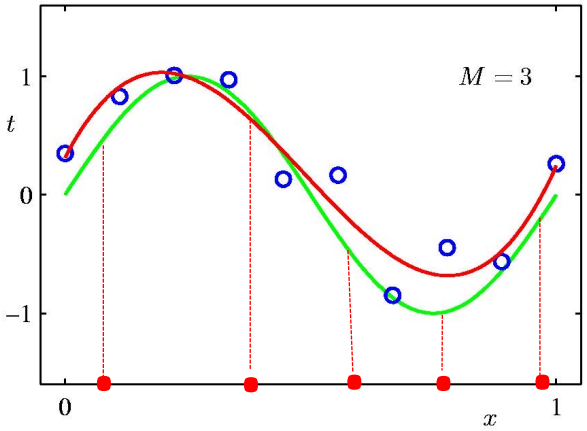


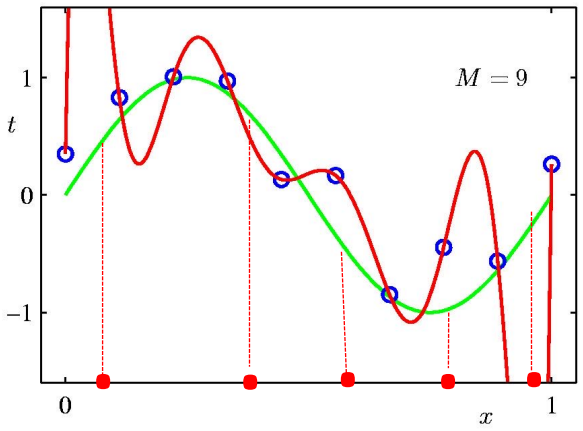


Sample *test* points:
Unseen by the learning algorithm, not in D!



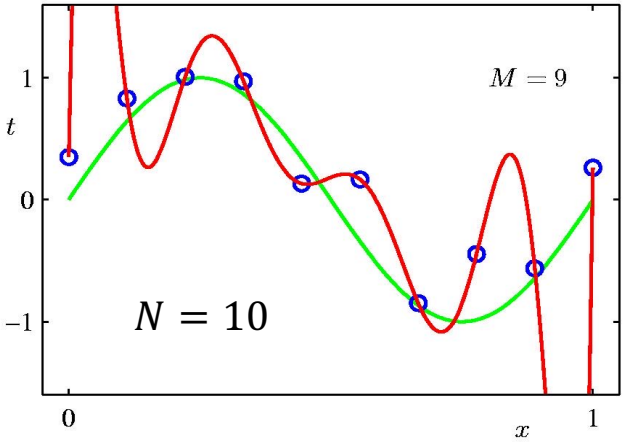






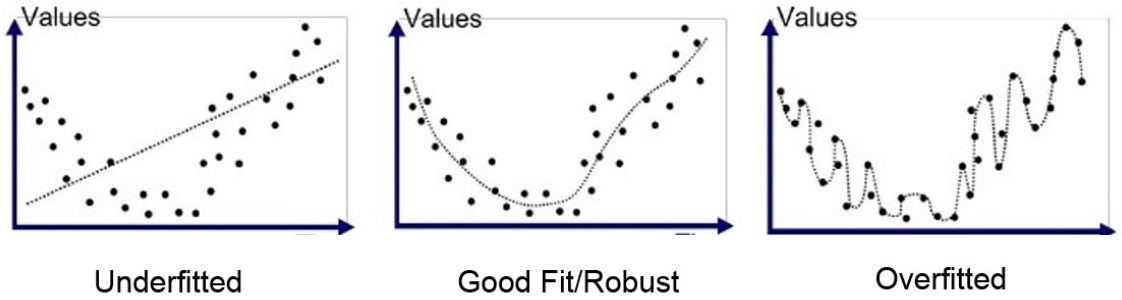
Overfitting vs. Generalization

Model: 9th order polynomial



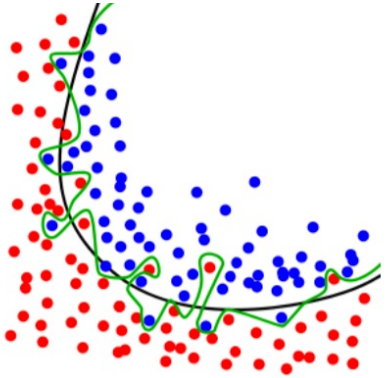
- ✓ RMSE = 0
- ❖ **Test error explodes!**
  - **Overfitting (learning the noise)**
  - Poor *generalization*
- ➢ Too little data for model's power: $M \geq m$
  - ➢ The model can perfectly fit each data point

Overfitting: In ML overfitting occurs when an algorithm fits too closely or even exactly to its training data, resulting in a model that can't make accurate predictions from any data other than the training data ☹

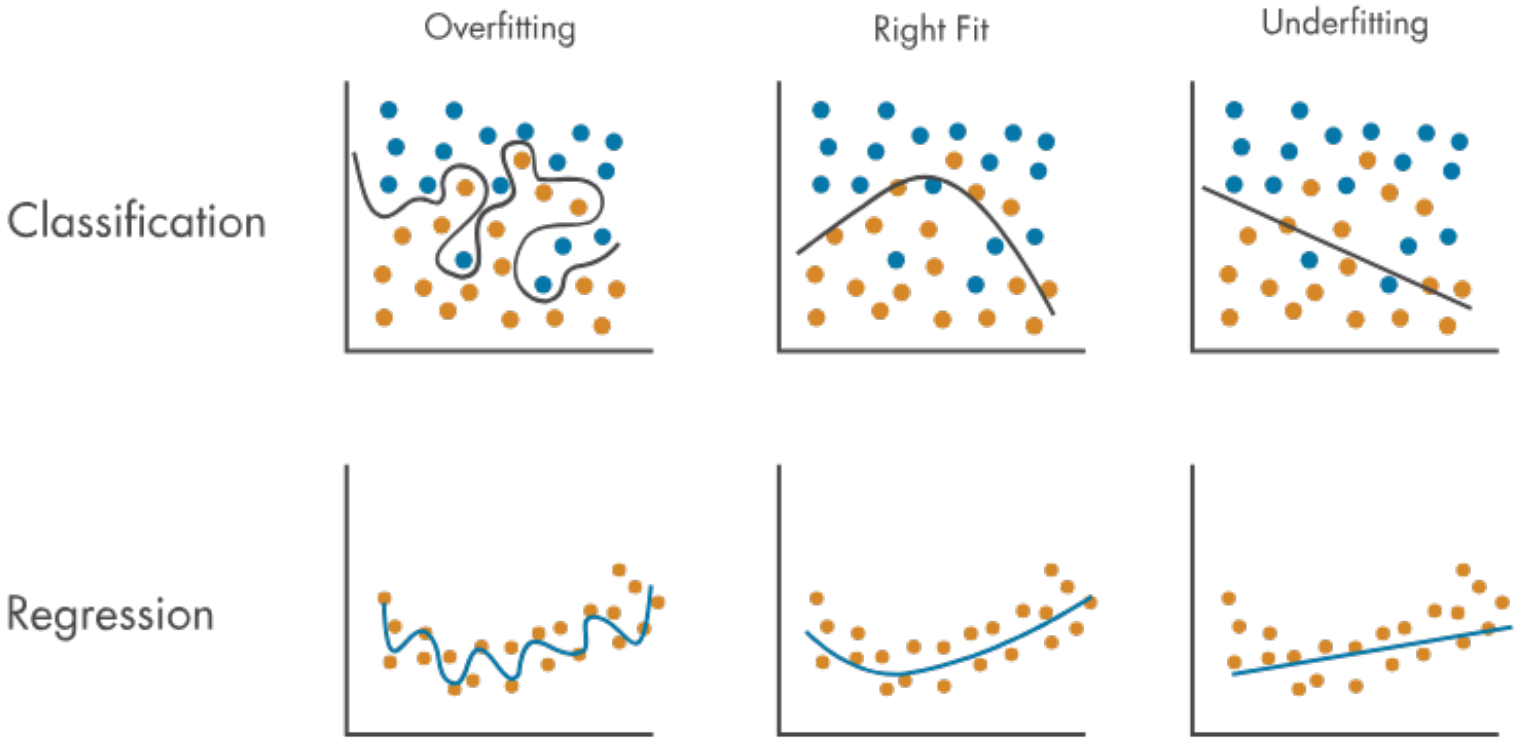


Overfitting defeats the purpose of ML since it doesn't support **generalization** of a model to (new) data outside of the training sample

Overfitting vs. Generalization

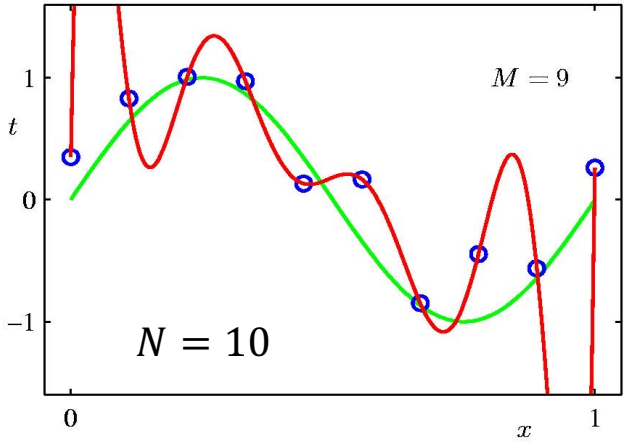


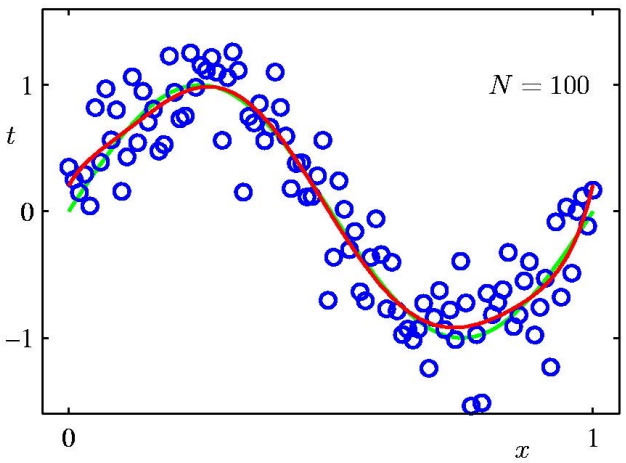
- Classification example: the green class separator brings to **overfitting**
- The black separator generalizes well



Overfitting vs. Generalization: Depends on Data + Model

9th order polynomial

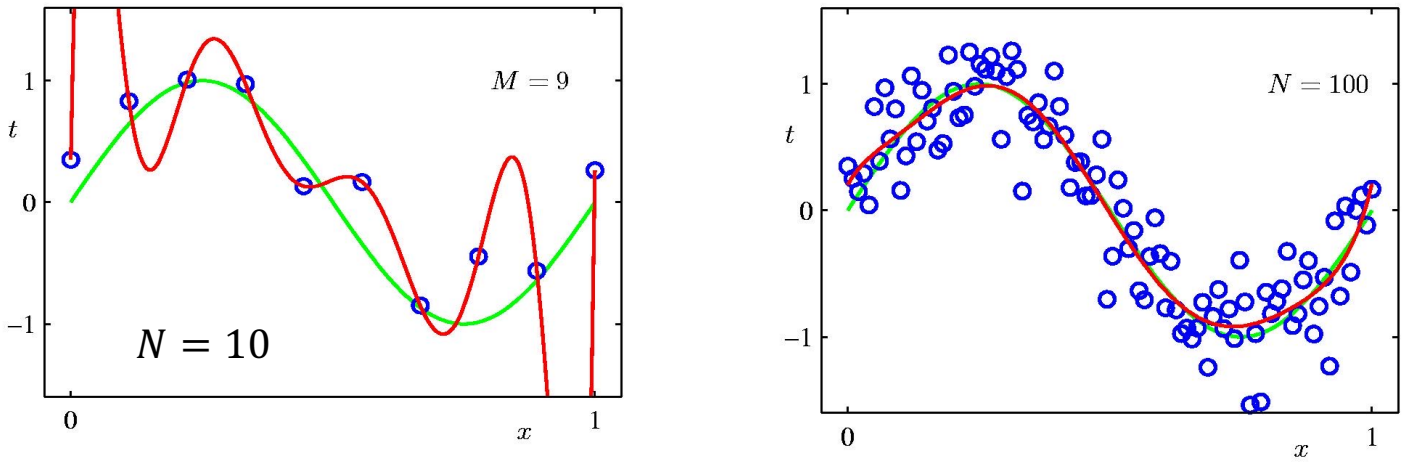




- ✓ RMSE = 0
- ❖ **Test error explodes!**
- ▪ **Overfitting (learning the noise)**
- ▪ Poor *generalization*
- ➢ Too little data for model's power: $M \geq m$

- ✓ Low RMSE
- ✓ Low test error
- ✓ No Overfitting
- ✓ Good generalization
- ➢ Way more data than parameters, $M < m$

Overfitting vs. Generalization



❖ Doing well on the training data (i.e., minimization of L_{emp}) is not enough for an ML algorithm

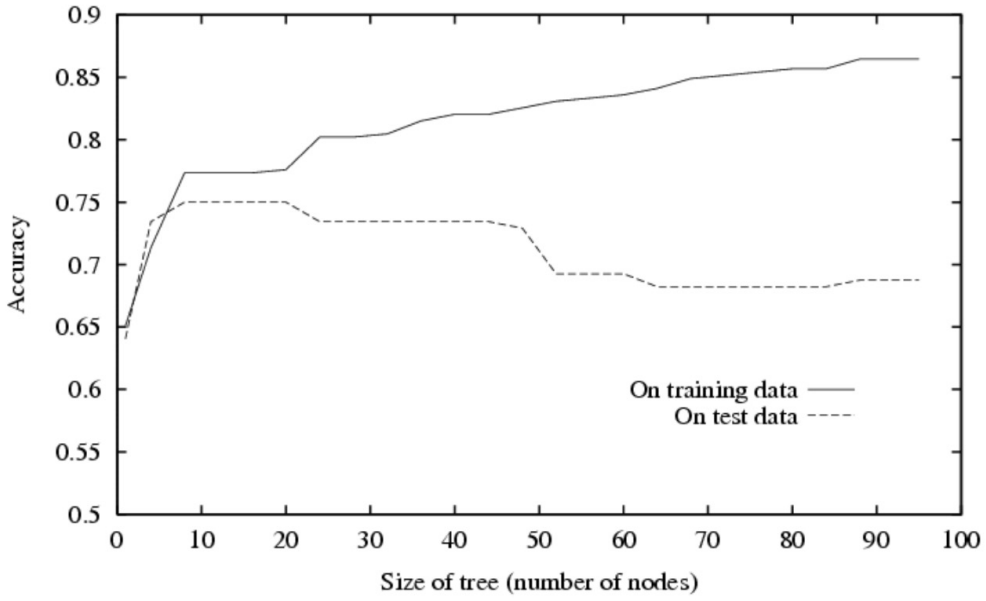
❖ Trying to do too well (or perfectly) on training data may lead to poor generalization

→ **Overfitting** (the training data)

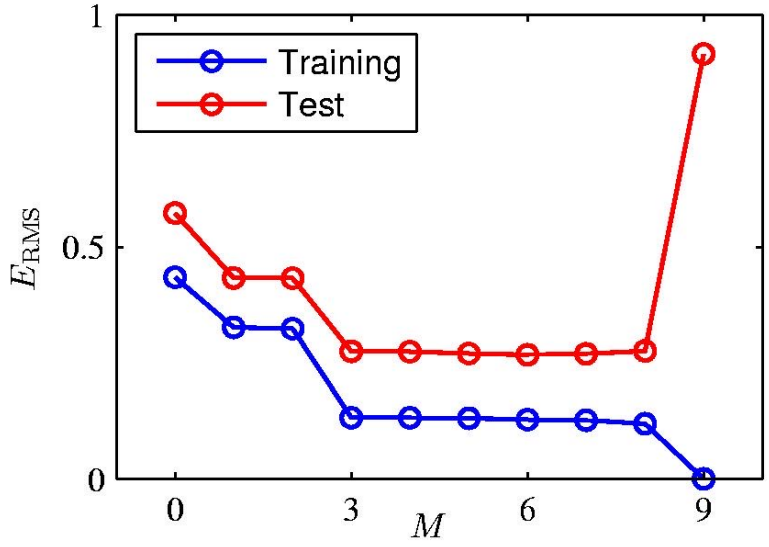
✓ Simple(r) models/functions tend to prevent overfitting and generalize well:
A key principle in designing ML algorithms (→ Regularization)

Which model complexity?

- ✓ By increasing the complexity / capacity of a model, we (ideally) expect a monotonic decrease in the training error.
- ➢ However, we also expect first a **decrease**, than a rapid **increase** of the validation (i.e., generalization) error, since we are most likely overfitting the training dataset

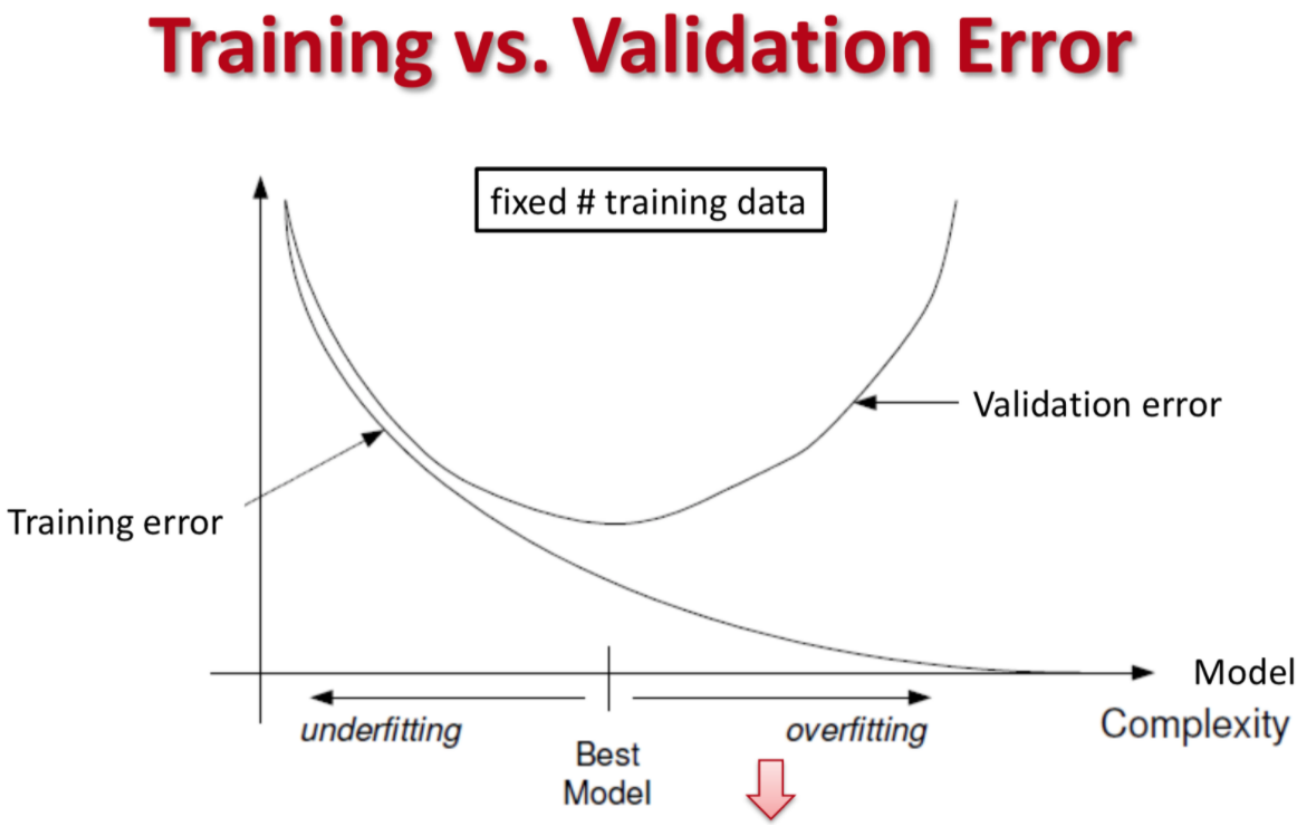


Task: learning which medical patients have a form of diabetes.



Which model complexity?

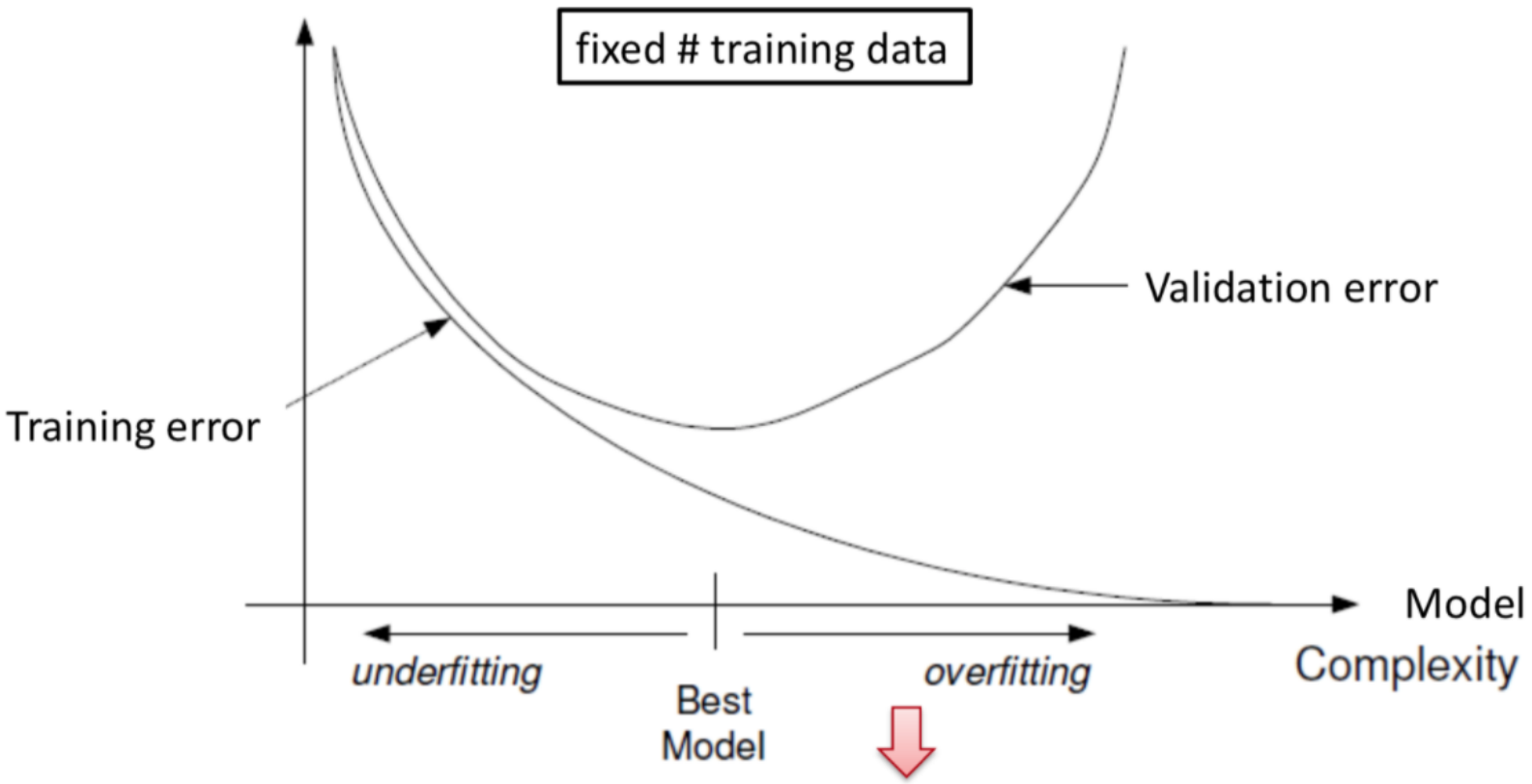
❖ Which model complexity (# parameters) is the best?



➢ That corresponding to the sweet spot where the validation error is about its minimum and the training error is quite low (yet, not zero)

Which model complexity?

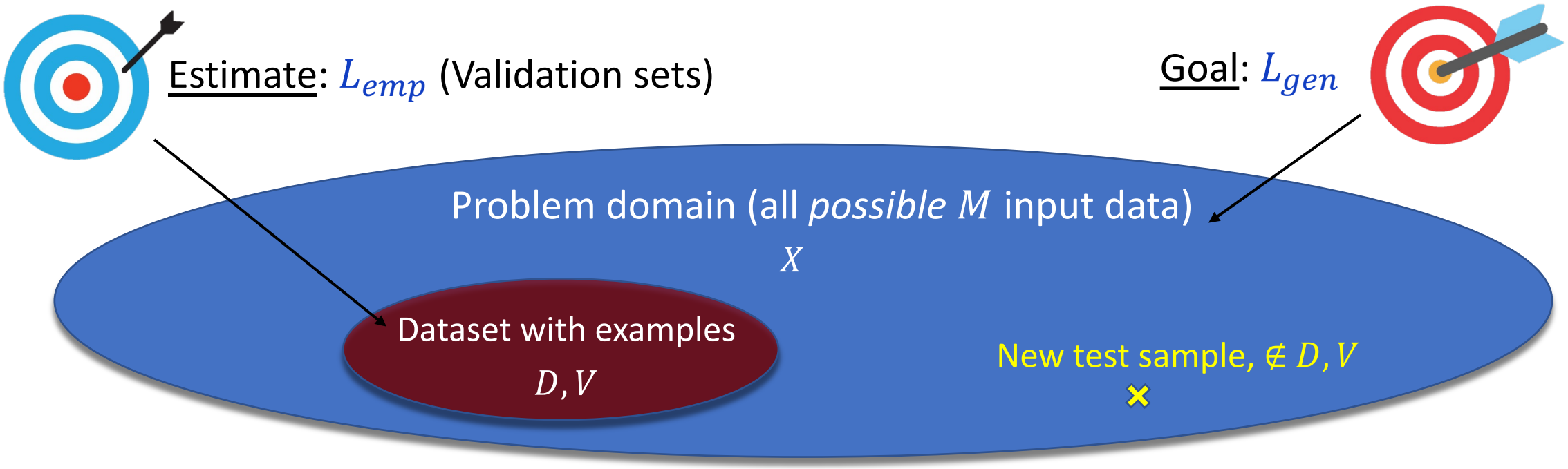
Training vs. Validation Error



Training error is no longer a
good indicator of validation or test error

Empirical Loss as an estimator for the Generalization Loss

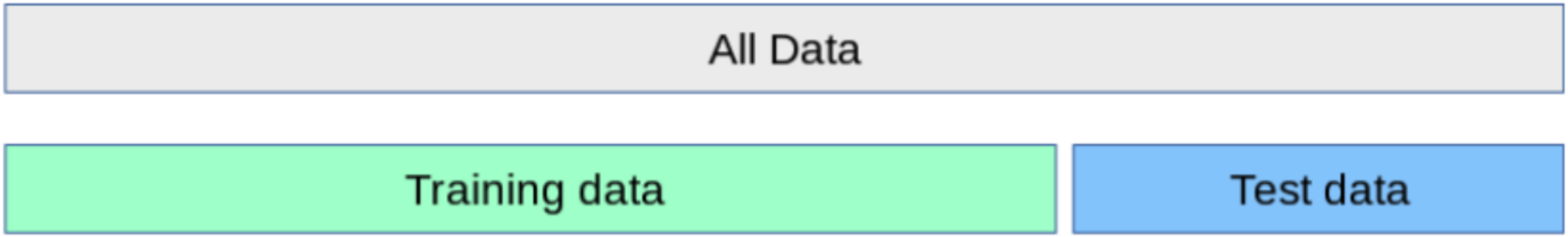
- $L_{emp}(D)$ on the **training set** D is a **local indicator** of L_{gen}, it drives the learning process
- $L_{emp}(V)$ on the **validation sets** V is used as an empirical estimate of L_{gen}



Hold-Out Method for defining Validation set(s)

Hold-out method → Break up the data in two groups:

- **Training** dataset, D, that we use to train the model
- **Testing** dataset, T, that we use to test/validate the model



✓ Use the empirical error on D to adapt model's parameters

✓ Use the empirical error on T as an estimate of the generalization error → Use it to revise the model (e.g., select a new model) or adapt it's (hyper)parameters, to reduce the expected generalization error

Hold-Out Method

Main rationale: evaluate the model using a set of data independent from the data used for training

Hold - out procedure:

n data points available $D \equiv \{X_i, Y_i\}_{i=1}^{n}$

Fundamental!

1) Randomly split into two sets (preserving label proportion):

Training dataset Validation/Hold-out dataset

$$D_T = \{X_i, Y_i\}_{i=1}^{m} \qquad D_V = \{X_i, Y_i\}_{i=m+1}^{n}$$

often m = n/2

2) Train classifier on D_T. Report error on validation dataset D_V.
Overfitting if validation error is much larger than training error

Hold-Out Method

1. Which **percentage of data** should be used for training / testing?
   - Maybe something like 70/30?
   - 50 / 50 ?

2. How should we **select the data** in the two groups?
   - **Randomly** seems a good criterion!
   - Just split the dataset arrays at the desired point, if data is already shuffled up

Hold-Out Method

Drawbacks:

- We may not have enough data to afford getting one subset aside (all these data examples won't be used for training!)
- If we get un unfortunate (or fortunate) split, the validation error might be a misleading estimate of the generalization error. The variance of the estimate can be large and depends on the size of the test set.

What to do?

- ✓ Hold-out method is computationally efficient, doesn't require additional computations
- ❖ However, it's limitations can be overcome by a family of sub-sampling methods (Cross-Validation), at the expenses of more computation

Cross-Validation: Validation sets without segregating data

Cross-Validation methods:

repeated *sub-sampling* of the available data for splitting in training and validation sets

❖ **Cross-validation** generalizes the concept of hold-out, by **iterating** the hold-out procedure K times, selecting each time different subsets for playing training and validation roles, without segregating the data in different roles!

✓ Under certain conditions (and relatively large datasets) CV not only provides an **empirical way to estimate the validation / generalization error,** but it also comes with formal proofs and guarantees about the **bounds of the empirical estimate with respect to the true (unknown) generalization error.**

➢ Splitting in training and validation sets can be done in many different ways, each with different pros and cons in terms of computational complexity and accuracy of the generalization error estimate.

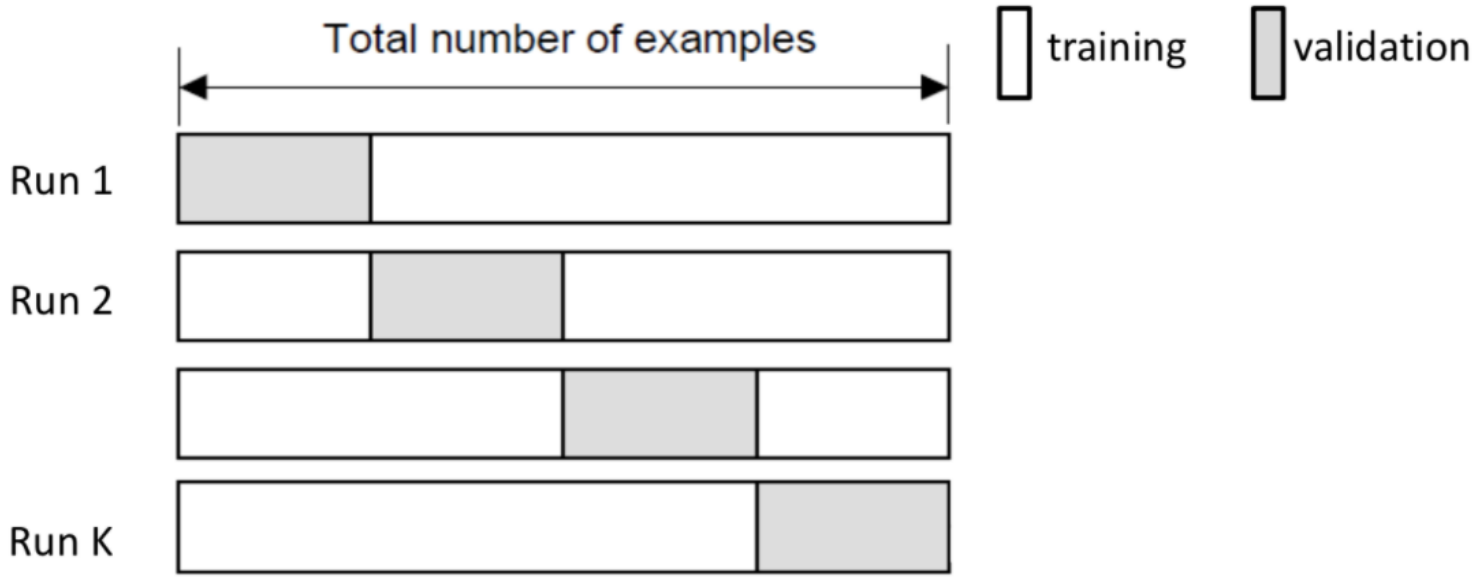
✓ The use of cross-validation methods is fundamental for **parameter optimization!**

K-fold Cross-Validation

K-Fold Cross-Validation

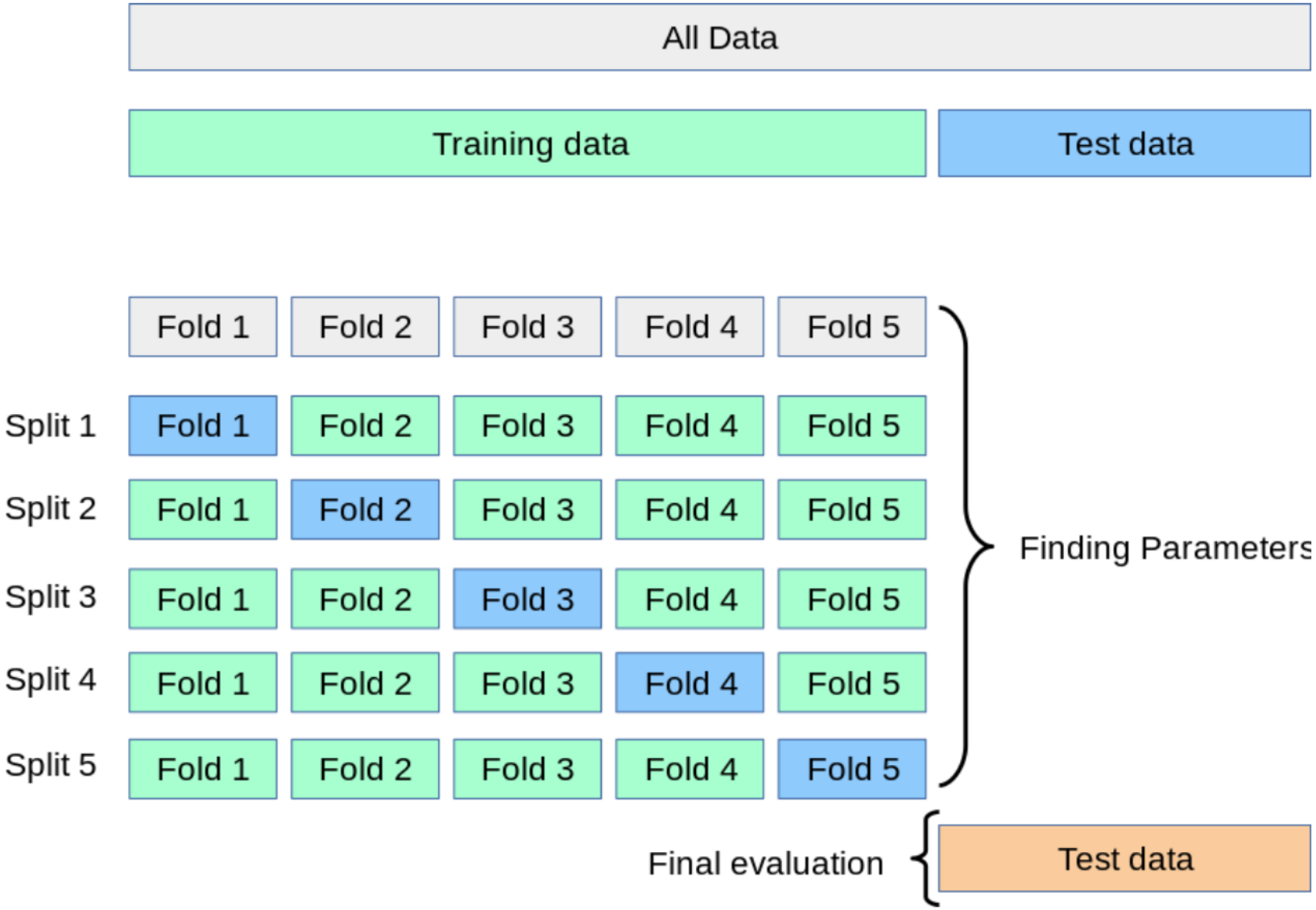
K-fold cross-validation

Create K-fold partition of the dataset.
Do K runs: train using K-1 partitions and calculate validation error on remaining partition (rotating validation partition on each run).
Report average validation error



K-fold Cross-Validation

Example of use with K =5 partitions.

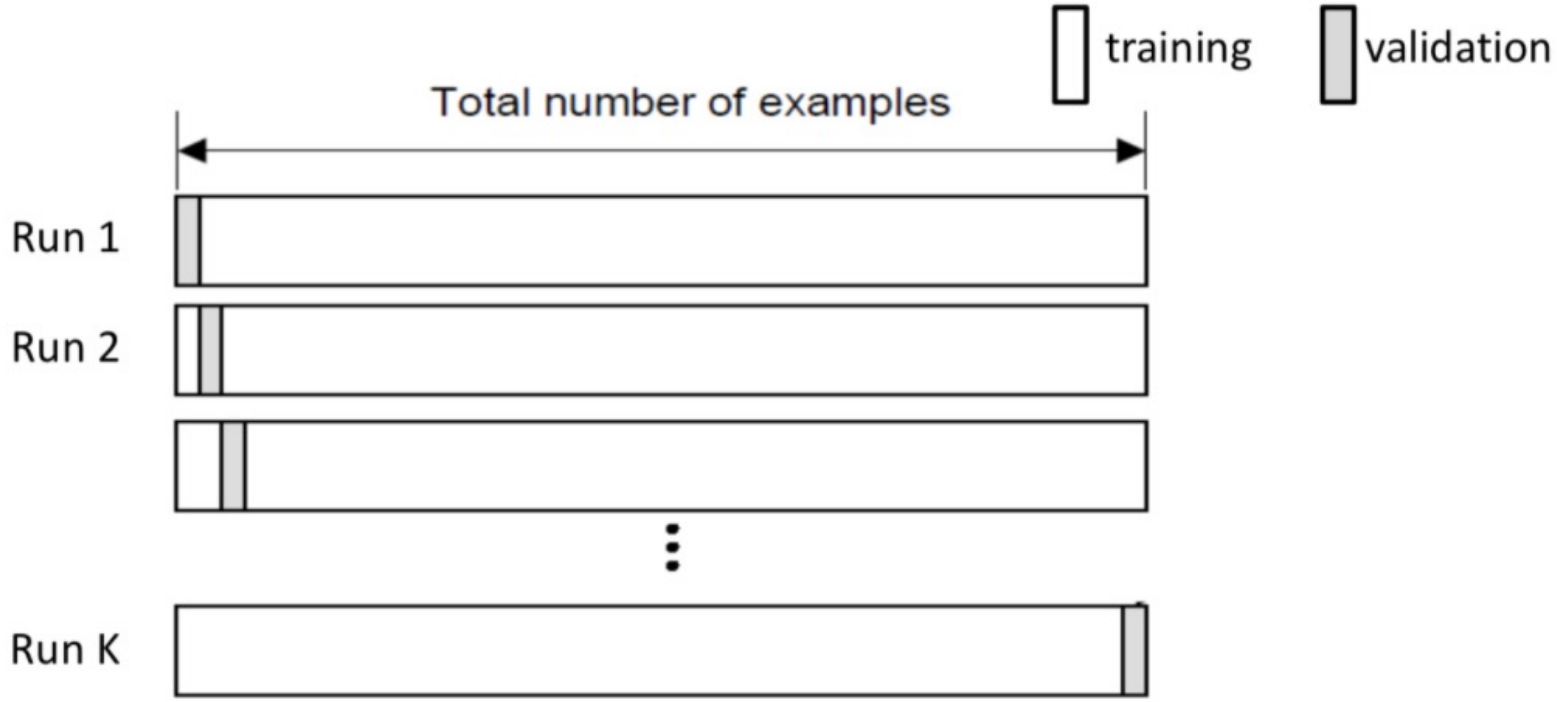


Leave-One-Out Cross-Validation

Leave-one-out (LOO) cross-validation

Special case of K-fold with K=n partitions

Equivalently, train on n-1 samples and validate on only one sample per run for n runs



Random Subsampling Cross-Validation

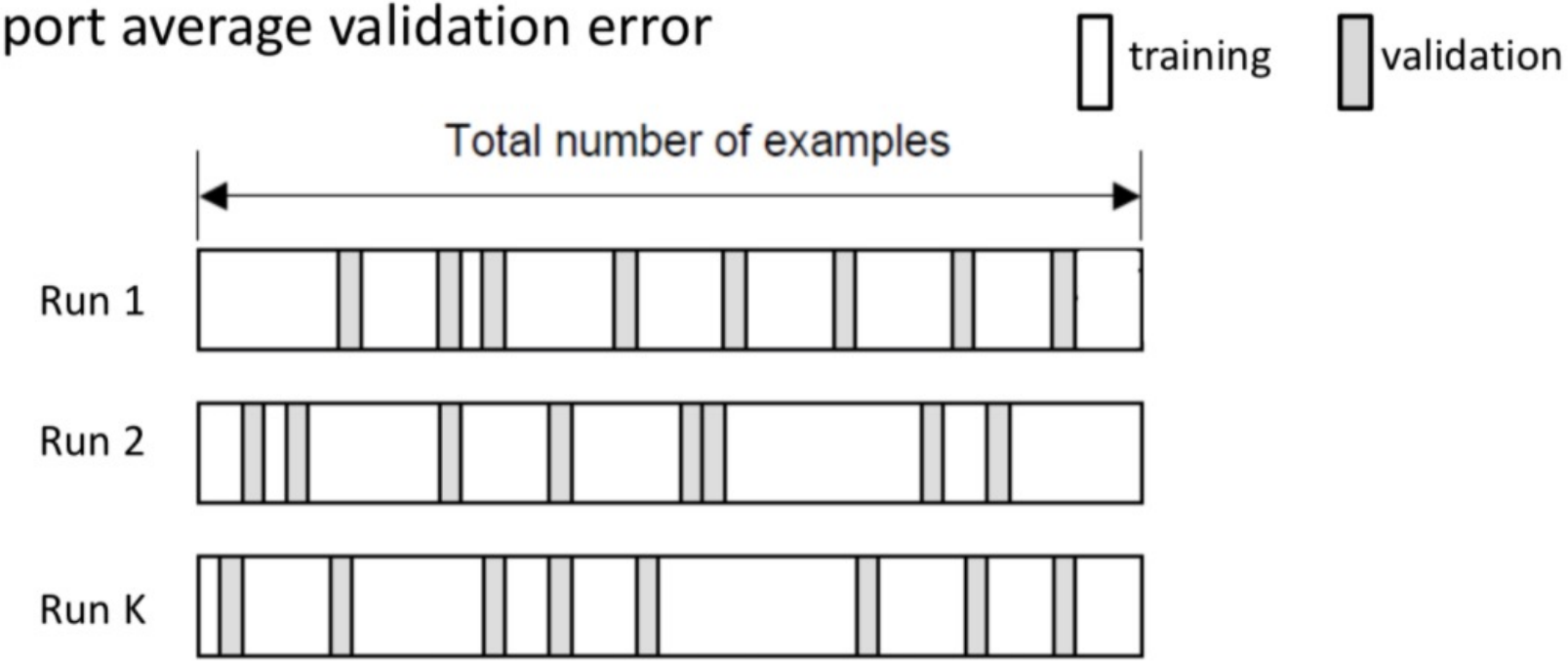
Random subsampling

Randomly subsample a fixed fraction αn ($0 < \alpha < 1$) of the dataset for validation.

Compute validation error with remaining data as training data.

Repeat K times

Report average validation error



How to select number of runs/partitions?

How to decide the values for K and α?

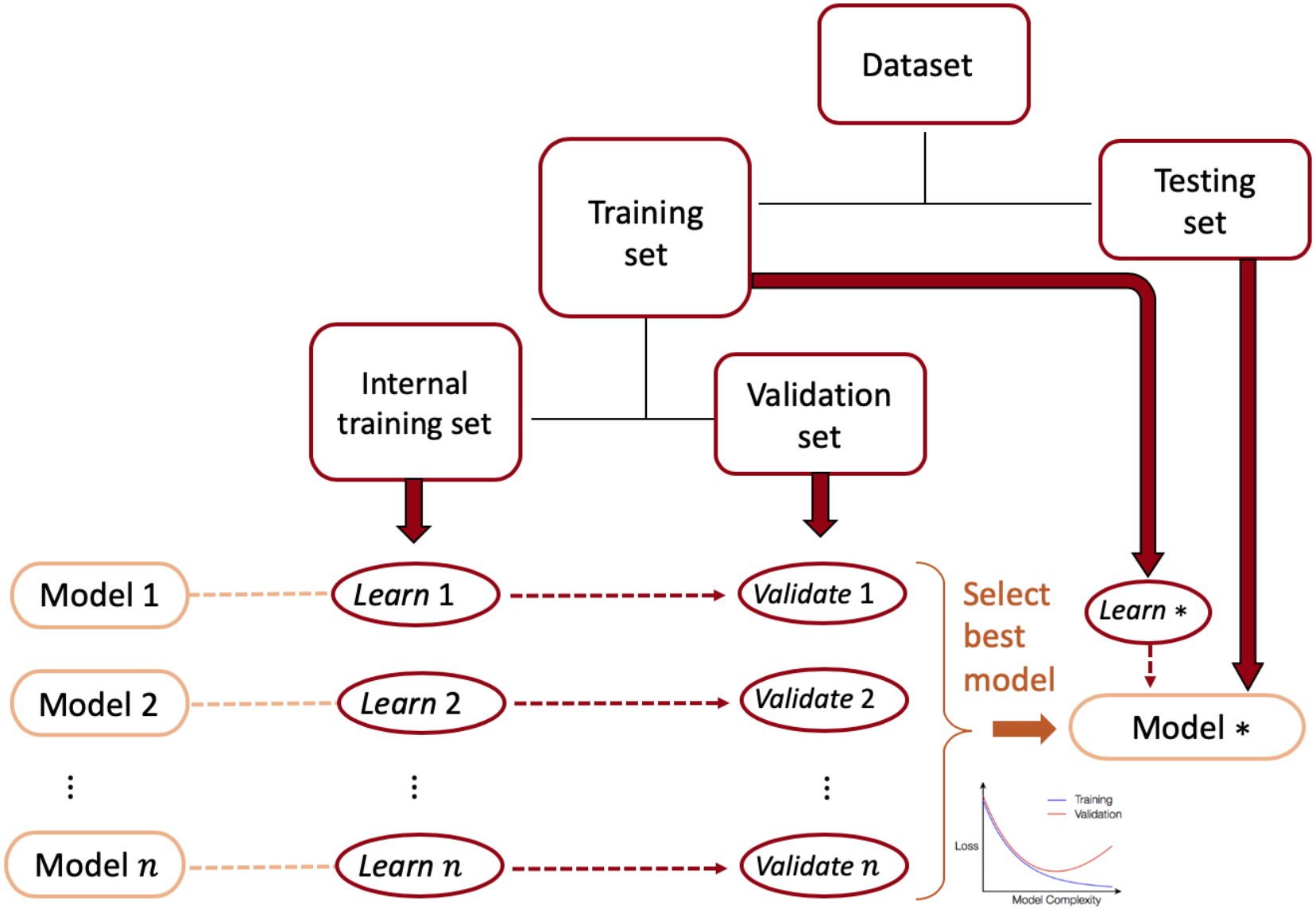
- Large K
  - + Validation error can approximate test error well
  - - Observed validation error will be unstable (few validation pts)
  - - The computational time will be very large as well (many experiments)

- Small K
  - + The # experiments and, therefore, computation time are reduced
  - + Observed validation error will be stable (many validation pts)
  - - Validation error cannot approximate test error well

Common choice: K = 10, α = 0.1 ☺

General view of model training, validation, optimization, selection, testing

- If the dataset is relatively large, it is initially split in a **Training set** and a **Testing set** (Hold-Out)
- The Training set is further split (possibly following a *Cross-Validation* approach) in:
  1. **Internal training set** (the actual set used for training)
  2. **Validation set(s)** (used for estimating the generalization error).
- IF multiple candidate models (either in terms of hyperparameters or learning algorithms) are available, each model goes through a joint training AND validation process.
  - Out of all candidate models, the **best is selected** (the one with minimal expected generalization error as estimated via the validation set)
    - The selected model is then **retrained**, using this time the **entire Training set**.
  - Finally, the trained and optimized selected model is tested on the **Testing set** (e.g., the one provided by the stakeholder or the one that was held out at the beginning)
- IF the result is not satisfactory (e.g., the error is much larger than the so far estimated generalization error), the process can be **restarted** (e.g., by selecting different candidate models).

✓ Otherwise, the selected model is accepted and *deployed*.

General view of model training, validation, optimization, selection, testing